

PL/SQL PROGRAMMING

UIN-66

PL/SQL

- PL/SQL stands for “Procedural Language extensions to the Structured Query Language.”
- Oracle Corporation introduced PL/SQL to overcome some limitations in SQL and to provide a more complete programming solution for those who sought to build mission-critical applications to run against the Oracle database
- It was developed by Oracle Corporation in the early 90's to enhance the capabilities of SQL. PL/SQL is one of three key programming languages embedded in the Oracle Database, along with SQL itself and Java.

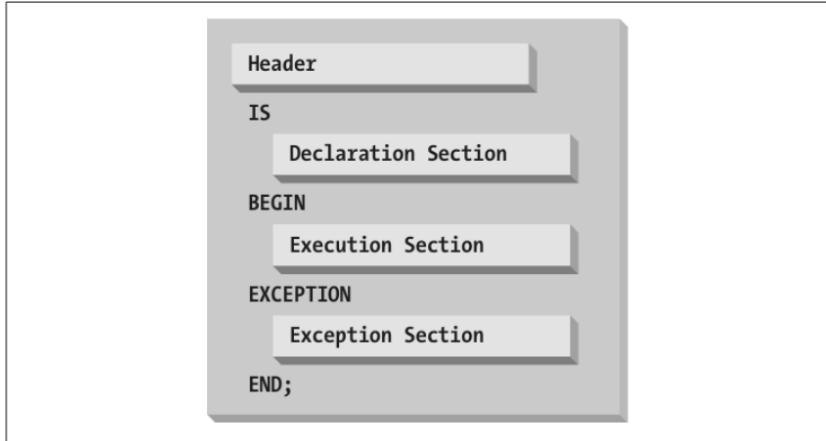


ADVANTAGES

- SQL is the standard database language and PL/SQL is strongly integrated with SQL. PL/SQL supports both static and dynamic SQL. Static SQL supports DML operations and transaction control from PL/SQL block. In Dynamic SQL, SQL allows embedding DDL statements in PL/SQL blocks.
 - PL/SQL allows sending an entire block of statements to the database at one time. This reduces network traffic and provides high performance for the applications.
 - PL/SQL gives high productivity to programmers as it can query, transform, and update data in a database.
 - PL/SQL saves time on design and debugging by strong features, such as exception handling, encapsulation, data hiding, and object-oriented data types.
 - Applications written in PL/SQL are fully portable.
 - PL/SQL provides high security level.
 - PL/SQL provides access to predefined SQL packages.
 - PL/SQL provides support for Object-Oriented Programming.
 - PL/SQL provides support for developing Web Applications and Server Pages.
- 

BLOCK STRUCTURE

Figure 3-1 shows the structure of the PL/SQL block for a procedure.



The diagram illustrates the structure of a PL/SQL block for a procedure. It is a vertical sequence of elements enclosed in a large gray box. The elements are: **Header** (a gray box), **IS** (text), **Declaration Section** (a gray box), **BEGIN** (text), **Execution Section** (a gray box), **EXCEPTION** (text), **Exception Section** (a gray box), and **END;** (text).

Figure 3-1. The PL/SQL block structure

Figure 3-2 shows a procedure containing all four sections of the elements of a block. This particular block begins with the keyword **PROCEDURE**, and, like all blocks, ends with the keyword **END**.

EXAMPLE

DECLARE

message varchar2(20):= 'Hello, World!';

BEGIN dbms_output.put_line(message);

END;

/

Output:

Hello World

PL/SQL procedure successfully completed.



PL/SQL PROGRAM UNITS

A PL/SQL unit is any one of the following –

- PL/SQL block
- Function
- Package
- Package body
- Procedure
- Trigger
- Type
- Type body



VARIABLE DECLARATION IN PL/SQL

variable name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial value]

Eg. name varchar2(25);

Initializing Variables in PL/SQL

- The **DEFAULT** keyword
- The **assignment** operator

For example –

counter binary integer := 0;

greetings varchar2(20) DEFAULT 'Have a Good Day';



EXAMPLE

DECLARE

a integer := 10;

b integer := 20;

c integer; f real;

BEGIN

c := a + b;

dbms_output.put_line('Value of c: ' || c);

f := 70.0/3.0;

dbms_output.put_line('Value of f: ' || f);

END;

/

Value of c: 30

Value of f: 23.33333333333333333333

PL/SQL procedure successfully completed.

VARIABLE SCOPE IN PL/SQL

Local variables – Variables declared in an inner block and not accessible to outer blocks.

Global variables – Variables declared in the outermost block or a package.



EXAMPLE

```
DECLARE
```

```
-- Global variables
```

```
num1 number := 95;
```

```
num2 number := 85;
```

```
BEGIN
```

```
dbms_output.put_line('Outer Variable num1: ' || num1); dbms_output.put_line('Outer Variable num2: ' || num2);
```

```
DECLARE
```

```
-- Local variables
```

```
num1 number := 195; num2 number := 185;
```

```
BEGIN
```

```
dbms_output.put_line('Inner Variable num1: ' || num1); dbms_output.put_line('Inner Variable num2: ' || num2);
```

```
END;
```

```
END;
```

```
/
```

```
OUTPUT
```

```
Outer Variable num1: 95 Outer Variable num2: 85
```

```
Inner Variable num1: 195 Inner Variable num2: 185
```

```
PL/SQL procedure successfully completed.
```

ASSIGNING SQL QUERY RESULTS

SELECT INTO statement of SQL to assign values to PL/SQL variables.

DECLARE

c_id customers.id%type := 1;

c_name customers.name%type;

c_addr customers.address%type;

c_sal customers.salary%type;

BEGIN

SELECT name, address, salary INTO c_name, c_addr, c_sal FROM customers WHERE id
= c_id;

dbms_output.put_line ('Customer ' || c_name || ' from ' || c_addr || ' earns ' || c_sal);

END;

/

OUTPUT:

Customer Ramesh from Ahmedabad earns 2000

PL/SQL procedure completed successfully

DECLARING A CONSTANT

```
PI CONSTANT NUMBER := 3.141592654;
DECLARE
radius number(5,2);
dia number(5,2);
circumference number(7, 2);
area number (10, 2);
BEGIN
radius := 9.5;
dia := radius * 2;
circumference := 2.0 * pi * radius;
area := pi * radius * radius;
dbms_output.put_line('Radius: ' || radius);
dbms_output.put_line('Diameter: ' || dia);
dbms_output.put_line('Circumference: ' || circumference);
dbms_output.put_line('Area: ' || area); END; /
```

o/p

Radius: 9.5 Diameter: 19 Circumference: 59.69 Area: 283.53 PI/SQL procedure successfully completed.

PL/SQL OPERATORS

Arithmetic operators

Relational operators

Comparison operators

Logical operators

String operators



ACCEPTING USER INPUT DURING RUNTIME:

1.

```
SQL> accept x number prompt 'Please enter something: '
```

Please enter something: 2

```
SQL> declare
```

```
2  a number;
```

```
3  begin
```

```
4  a := &x;
```

```
5  end;
```

```
6  /
```

```
old 4: a := &x;
```

```
new 4: a :=      2;
```

PL/SQL procedure successfully completed.

SQL> accept x char prompt 'Please enter something: '

Please enter something: sam

SQL> declare

2 a varchar2(10);

3 begin

4 a := '&x'; -- for a substitution variable of char data type

5 end; -- to be treated as a character string it needs

6 /

old 4: a := '&x'; -- for a substitution variable of char data type

new 4: a := 'sam'; -- for a substitution variable of char data type

PL/SQL procedure successfully completed.

2.

SQL> declare

y number;

begin

y:=&y;

-- Below line will output the number you received as an input

dbms_output.put_line(y);

end;

/

Enter value for y: 5

old 4: y:=&y;

new 4: y:=5;

PL/SQL procedure successfully completed.

PL/SQL STATEMENTS

S.No	Statement & Description
1	<u>IF - THEN statement</u> The IF statement associates a condition with a sequence of statements enclosed by the keywords THEN and END IF . If the condition is true, the statements get executed and if the condition is false or NULL then the IF statement does nothing.
2	<u>IF-THEN-ELSE statement</u> IF statement adds the keyword ELSE followed by an alternative sequence of statement. If the condition is false or NULL, then only the alternative sequence of statements get executed. It ensures that either of the sequence of statements is executed.
3	<u>IF-THEN-ELSIF statement</u> It allows you to choose between several alternatives.
4	<u>Case statement</u> Like the IF statement, the CASE statement selects one sequence of statements to execute. However, to select the sequence, the CASE statement uses a selector rather than multiple Boolean expressions. A selector is an expression whose value is used to select one of several alternatives.
5	<u>Searched CASE statement</u> The searched CASE statement has no selector , and it's WHEN clauses contain search conditions that yield Boolean values.
6	<u>nested IF-THEN-ELSE</u> You can use one IF-THEN or IF-THEN-ELSIF statement inside another IF-THEN or IF-THEN-ELSIF statement(s).

EXAMPLE

DECLARE

a number(3) := 100;

BEGIN

IF (a = 10) THEN

dbms_output.put_line('Value of a is 10');

ELSIF (a = 20) THEN

dbms_output.put_line('Value of a is 20');

ELSIF (a = 30) THEN

dbms_output.put_line('Value of a is 30');

ELSE

dbms_output.put_line('None of the values is matching');

END IF;

dbms_output.put_line('Exact value of a is: ' || a);

END;

/

o/p

None of the values is matching

Exact value of a is: 100

PL/SQL procedure successfully completed.

Example: Find out maximum no between two .



PL/SQL Control Statement Exercises: Check whether a number is even or odd

```
DECLARE
n1 NUMBER := &num1;
BEGIN
-- test if the number provided by the user is even
IF MOD(n1,2) = 0 THEN
DBMS_OUTPUT.PUT_LINE ('The number. ' || n1 || ' is even number');
ELSE
DBMS_OUTPUT.PUT_LINE ('The number ' || n1 || ' is odd number. ');
END IF;
DBMS_OUTPUT.PUT_LINE ('Done Successfully');
END;
/
```

PL/SQL Control Statement Exercises: Check whether a given number is positive, negative or zero

```
DECLARE
num1 NUMBER := &get_num;
BEGIN IF num1 < 0 THEN
DBMS_OUTPUT.PUT_LINE ('The number ' || num1 || ' is a negative number');
ELSIF num1 = 0 THEN
DBMS_OUTPUT.PUT_LINE ('The number ' || num1 || ' is equal to zero');
ELSE DBMS_OUTPUT.PUT_LINE ('The number ' || num1 || ' is a positive number'); END
    IF;
END; /
```



PL/SQL Control Statement Exercises: Convert a temperature in scale Fahrenheit to Celsius and vice versa

DECLARE

temp1 NUMBER := &input_a_temp;

t_scale CHAR := '&input_temp_scale';

new_temp NUMBER;

new_scale CHAR;

BEGIN

IF t_scale != 'C' AND t_scale != 'F' THEN

dbms_output.Put_line ('The scale you input is not a valid scale');

new_temp := 0;

new_scale := 'C';

ELSE IF t_scale = 'C' THEN

new_temp := ((9 * temp1) / 5) + 32;

new_scale := 'F';

ELSE new_temp := ((temp1 - 32) * 5) / 9;

new_scale := 'C';

END IF;

END IF;

dbms_output.Put_line ('The new temperature in scale ' || new_scale || ' is: ' || new_temp); END; /

CASE STATEMENT

```
DECLARE grade char(1) := 'A';  
BEGIN  
CASE grade  
when 'A' then dbms_output.put_line('Excellent');  
  when 'B' then dbms_output.put_line('Very good');  
when 'C' then dbms_output.put_line('Well done');  
when 'D' then dbms_output.put_line('You passed');  
  when 'F' then dbms_output.put_line('Better try again');  
  else dbms_output.put_line('No such grade');  
END CASE;  
END;
```



PL/SQL LOOP STATEMENTS

S.No	Loop Type & Description
1	<u>PL/SQL Basic LOOP</u> In this loop structure, sequence of statements is enclosed between the LOOP and the END LOOP statements. At each iteration, the sequence of statements is executed and then control resumes at the top of the loop.
2	<u>PL/SQL WHILE LOOP</u> Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
3	<u>PL/SQL FOR LOOP</u> Execute a sequence of statements multiple times and abbreviates the code that manages the loop variable.
4	<u>Nested loops in PL/SQL</u> You can use one or more loop inside any another basic loop, while, or for loop.


```
DECLARE
  i number(1);
  j number(1);
BEGIN
  << outer_loop >>
  FOR i IN 1..3 LOOP
    << inner_loop >>
    FOR j IN 1..3 LOOP
      dbms_output.put_line('i is:' || i || ' and j is:' || j);
    END loop inner_loop;
  END loop outer_loop;
END;
```

/

i is: 1 and j is: 1 i is: 1 and j is: 2 i is: 1 and j is: 3 i is: 2 and j is: 1 i is: 2 and j is: 2 i is: 2 and j is: 3 i is: 3 and j is: 1 i is: 3 and j is: 2 i is: 3 and j is: 3 PL/SQL procedure successfully completed.

DECLARE

a number(2) := 10;

BEGIN

WHILE a < 20 LOOP

dbms_output.put_line('value of a: ' || a);

a := a + 1;

END LOOP;

END;

/

value of a: 10 value of a: 11 value of a: 12 value of a: 13 value of a: 14 value of a: 15
value of a: 16 value of a: 17 value of a: 18 value of a: 19 PL/SQL procedure
successfully completed.

```
DECLARE
```

```
v NUMBER(1):=1;
```

```
BEGIN
```

```
--<<outer_loop>>
```

```
FOR v IN (SELECT students.student_name FROM students WHERE Marks>=100)
```

```
LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('student name who score more than 100  
    is' || v.student_name);
```

```
END LOOP;
```

```
END;
```

```
/
```



THE LOOP CONTROL STATEMENTS

S.No	Control Statement & Description
1	<u>EXIT statement</u> The Exit statement completes the loop and control passes to the statement immediately after the END LOOP.
2	<u>CONTINUE statement</u> Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
3	<u>GOTO statement</u> Transfers control to the labeled statement. Though it is not advised to use the GOTO statement in your program.

```
DECLARE
a number(2) := 10;
BEGIN
-- while loop execution
WHILE a < 20
LOOP
dbms_output.put_line ('value of a: ' || a);
a := a + 1;
IF a > 15 THEN
-- terminate the loop using the exit statement
EXIT;
END IF;
END LOOP;
END; /
```

o/p:

value of a: 10 value of a: 11 value of a: 12 value of a: 13 value of a: 14 value of a: 15

DECLARE

a number(2) := 10;

BEGIN

-- while loop execution

WHILE a < 20

LOOP

dbms_output.put_line ('value of a: ' || a);

a := a + 1;

-- terminate the loop using the exit when statement

EXIT WHEN a > 15;

END LOOP;

END; /

o/p:

value of a: 10 value of a: 11 value of a: 12 value of a: 13 value of a: 14 value of a: 15

```
DECLARE
a number(2) := 10;
BEGIN
<<loopstart>>
  -- while loop execution
WHILE a < 20
  LOOP
dbms_output.put_line ('value
    of a: ' || a);
a := a + 1;
IF a = 15 THEN
a := a + 1;
GOTO loopstart;
ENDIF;
END LOOP;
END;/
```

o/p

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 16

value of a: 17

value of a: 18

value of a: 19

PL/SQL procedure successfully
completed.

RESTRICTIONS WITH GOTO STATEMENT

GOTO Statement in PL/SQL imposes the following restrictions –

- A GOTO statement cannot branch into an IF statement, CASE statement, LOOP statement or sub-block.
 - A GOTO statement cannot branch from one IF statement clause to another or from one CASE statement WHEN clause to another.
 - A GOTO statement cannot branch from an outer block into a sub-block (i.e., an inner BEGIN-END block).
 - A GOTO statement cannot branch out of a subprogram. To end a subprogram early, either use the RETURN statement or have GOTO branch to a place right before the end of the subprogram.
 - A GOTO statement cannot branch from an exception handler back into the current BEGIN-END block. However, a GOTO statement can branch from an exception handler into an enclosing block.
- 

LINK

<https://padlet.com/sampadamargaj/2w6a57dao272yzif>

How to save commands in separate file:

```
SQL>save myFile;
```

```
SQL>@myFile;
```

CASE STATEMENT

CASE (expression)

WHEN <value1> THEN action_block1;

WHEN <value2> THEN action_block2;

WHEN <value3> THEN action_block3;

ELSE action_block_default;

END CASE;



```
DECLARE
a NUMBER :=55;
b NUMBER :=5;
arth_operation VARCHAR2(20) :='MULTIPLY';
BEGIN
dbms_output.put_line('Program started. ');
CASE (arth_operation)
WHEN 'ADD' THEN
dbms_output.put_line('Addition of the numbers are: ' || a+b );
WHEN 'SUBTRACT' THEN dbms_output.put_line('Subtraction of the numbers are: ' || a-b);
WHEN 'MULTIPLY' THEN dbms_output.put_line('Multiplication of the numbers are: ' || a*b
);
WHEN 'DIVIDE' THEN dbms_output.put_line('Division of the numbers are: ' || a/b);
ELSE dbms_output.put_line('No operation action defined. Invalid operation');
END CASE;
dbms_output.put_line('Program completed. ');
END; /
o/p:Program started. Multiplication of the numbers are: 275 Program completed
```

USING THE %TYPE ATTRIBUTE

The %TYPE attribute provides the datatype of a variable or database column.

```
CREATE TABLE employees_temp (empid NUMBER(6) NOT NULL PRIMARY KEY, deptid  
    NUMBER(6) CONSTRAINT check_deptid CHECK (deptid BETWEEN 100 AND 200),  
    deptname VARCHAR2(30) DEFAULT 'Sales');
```

DECLARE

```
    v_empid employees_temp.empid%TYPE;
```

```
    v_deptid employees_temp.deptid%TYPE;
```

```
    v_deptname employees_temp.deptname%TYPE;
```

BEGIN

```
    Select deptname into v_deptname from employees_temp where empid=1;
```

```
    DBMS_OUTPUT.PUT_LINE('v_deptname: ' || v_deptname);
```

```
END;
```

```
/
```

USING THE %ROWTYPE ATTRIBUTE

The %ROWTYPE attribute provides a record type that represents a row in a table or view. Columns in a row and corresponding fields in a record have the same names and datatypes.

DECLARE

dept_rec departments%ROWTYPE;

BEGIN

**SELECT * INTO dept_rec FROM departments WHERE department_id = 30 and
ROWNUM < 2;**

END; /



```
SQL> CREATE TABLE students( Roll_no INT NOT NULL PRIMARY KEY,student_name  
    VARCHAR(100),Marks INT);
```

Table created.

```
DECLARE
```

```
v NUMBER(1):=1;
```

```
rec students%ROWTYPE;
```

```
BEGIN
```

```
--<<outer_loop>>
```

```
FOR rec IN (SELECT students.student_name FROM students WHERE Marks>=100)
```

```
LOOP
```

```
DBMS_OUTPUT.PUT_LINE('student name who score more than 100 is' || rec.student_name);
```

```
END LOOP;
```

```
END;
```

```
/
```

o/p

student name who score more than 100 isPranav

student name who score more than 100 isRohan

```
CREATE TABLE employees_temp
```

```
(empid NUMBER(6) NOT NULL PRIMARY KEY, deptid NUMBER(6) CONSTRAINT  
  c_employees_temp_deptid CHECK (deptid BETWEEN 100 AND 200), deptname  
  VARCHAR2(30) DEFAULT 'Sales' );
```

Table created.

```
DECLARE
```

```
emprec employees_temp%ROWTYPE;
```

```
BEGIN
```

```
SELECT * INTO emprec FROM employees_temp WHERE empid = 1;
```

```
DBMS_OUTPUT.PUT_LINE ('emprec.deptname: ' || emprec.deptname);
```

```
END;
```

```
/
```

```
emprec.deptname: Accounts
```

```
PL/SQL procedure successfully completed.
```

STRINGS

The string in PL/SQL is actually a sequence of characters with an optional size specification. The characters could be numeric, letters, blank, special characters or a combination of all. PL/SQL offers three kinds of strings –

Fixed-length strings – In such strings, programmers specify the length while declaring the string. The string is right-padded with spaces to the length so specified.

Variable-length strings – In such strings, a maximum length up to 32,767, for the string is specified and no padding takes place.

Character large objects (CLOBs) – These are variable-length strings that can be up to 128 terabytes.

PL/SQL strings could be either variables or literals. A string literal is enclosed within quotation marks. For example,

'This is a string literal.' Or 'hello world'

To include a single quote inside a string literal, you need to type two single quotes next to one another. For example,

'this isn't what it looks like'

DECLARE

name varchar2(20);

company varchar2(30);

introduction clob;

choice char(1);

BEGIN

name := 'John Smith';

company := 'Infotech';

introduction := ' Hello! I'm John Smith from Infotech.';

choice := 'y';

IF choice = 'y' THEN

dbms_output.put_line(name);

dbms_output.put_line(company);

dbms_output.put_line(introduction);

END IF;

END; /

S.No	Function & Purpose
1	ASCII(x); Returns the ASCII value of the character x.
2	CHR(x); Returns the character with the ASCII value of x.
3	CONCAT(x, y); Concatenates the strings x and y and returns the appended string.
4	INITCAP(x); Converts the initial letter of each word in x to uppercase and returns that string.
5	INSTR(x, find_string [, start] [, occurrence]); Searches for find_string in x and returns the position at which it occurs.
6	INSTRB(x); Returns the location of a string within another string, but returns the value in bytes.
7	LENGTH(x); Returns the number of characters in x.
8	LENGTHB(x); Returns the length of a character string in bytes for single byte character set.
9	LOWER(x); Converts the letters in x to lowercase and returns that string.
10	LPAD(x, width [, pad_string]) ; Pads x with spaces to the left, to bring the total length of the string up to width characters.
11	LTRIM(x [, trim_string]); Trims characters from the left of x.
12	NANVL(x, value); Returns value if x matches the NaN special value (not a number), otherwise x is returned. NLS_INITCAP(x); Same as the INITCAP function except that it can use a different sort method as specified by NLSSORT.

NLS_LOWER(x) ;

Same as the LOWER function except that it can use a different sort method as specified by NLSSORT.

NLS_UPPER(x);

Same as the UPPER function except that it can use a different sort method as specified by NLSSORT.

NLSSORT(x);

Changes the method of sorting the characters. Must be specified before any NLS function; otherwise, the default sort will be used.

NVL(x, value);Returns value if x is null; otherwise, x is returned.

NVL2(x, value1, value2); Returns value1 if x is not null; if x is null, value2 is returned.

REPLACE(x, search_string, replace_string);

Searches x for search_string and replaces it with replace_string.

RPAD(x, width [, pad_string]);Pads x to the right.

RTRIM(x [, trim_string]);Trims x from the right.

SOUNDEX(x) ;Returns a string containing the phonetic representation of x.

SUBSTR(x, start [, length]);

Returns a substring of x that begins at the position specified by start. An optional length for the substring may be supplied.

SUBSTRB(x);

Same as SUBSTR except that the parameters are expressed in bytes instead of characters for the single-byte character systems.

TRIM([trim_char FROM] x);Trims characters from the left and right of x.

UPPER(x);Converts the letters in x to uppercase and returns that string.

DECLARE

greetings varchar2(11) := 'hello world';

BEGIN

dbms_output.put_line(UPPER(greetings));

dbms_output.put_line(LOWER(greetings)); dbms_output.put_line(INITCAP(greetings));

dbms_output.put_line (SUBSTR (greetings, 1, 1));

dbms_output.put_line (SUBSTR (greetings, -1, 1));

dbms_output.put_line (SUBSTR (greetings, 7, 5));

dbms_output.put_line (SUBSTR (greetings, 2));

dbms_output.put_line (INSTR (greetings, 'e'));

END;

/

HELLO WORLD

hello world

Hello World

h

d

World

Ello World

2

DECLARE

 str VARCHAR2(40) := 'Ramesh is a good boy';

 str1 VARCHAR2(40) := 'Suresh is a brilliant student';

 str2 VARCHAR2(40) := 'and';

 v VARCHAR2(100);

BEGIN

 v := str

 ||''

 || str2

 ||''

 ||str1;

 dbms_output.Put_line(v);

END;

DECLARE

s_name VARCHAR2(20);

s_age NUMBER(2);

s_address CLOB;

s_gender CHAR(1);

BEGIN

s_name := 'Sherlock Holmes';

s_age := 21;

s_address := '221B, Baker Street, London';

s_gender := 'M';

IF s_gender = 'M' THEN

dbms_output.put_line(s_name || ' is a student at Studytonight');

dbms_output.put_line('He is ' || s_age || ' years old.');

dbms_output.put_line('He lives at ' || s_address); ELSE

dbms_output.put_line(s_name || ' is a student at Studytonight');

dbms_output.put_line('She is ' || s_age || ' years old.');

dbms_output.put_line('She lives at ' || s_address);

END IF; END; /

PL/SQL PROCEDURES AND FUNCTIONS

A **subprogram** is a program unit/module that performs a particular task. These subprograms are combined to form larger programs. This is basically called the 'Modular design'. A subprogram can be invoked by another subprogram or program which is called the **calling program**.

A subprogram can be created –

At the schema level

Inside a package

Inside a PL/SQL block

At the schema level, subprogram is a **standalone subprogram**. It is created with the CREATE PROCEDURE or the CREATE FUNCTION statement. It is stored in the database and can be deleted with the DROP PROCEDURE or DROP FUNCTION statement.

A subprogram created inside a package is a **packaged subprogram**. It is stored in the database and can be deleted only when the package is deleted with the DROP PACKAGE statement. We will discuss packages in the chapter '**PL/SQL - Packages**'.

PL/SQL subprograms are named PL/SQL blocks that can be invoked with a set of parameters. PL/SQL provides two kinds of subprograms –

Functions – These subprograms return a single value; mainly used to compute and return a value.

Procedures – These subprograms do not return a value directly; mainly used to perform an action.

CREATING A PROCEDURE

A procedure is created with the **CREATE OR REPLACE PROCEDURE** statement.

```
CREATE [OR REPLACE] PROCEDURE procedure_name [(parameter_name [IN | OUT | IN  
    OUT] type [, ...])] {IS | AS}  
BEGIN < procedure_body >  
END procedure_name;
```

The optional parameter list contains name, mode and types of the parameters. **IN** represents the value that will be passed from outside and **OUT** represents the parameter that will be used to return a value outside of the procedure.

The **AS** keyword is used instead of the **IS** keyword for creating a standalone procedure.

Example

The following example creates a simple procedure that displays the string 'Hello World!' on the screen when executed.

```
CREATE OR REPLACE PROCEDURE greetings AS  
BEGIN dbms_output.put_line('Hello World!'); END;  
Procedure created.
```


EXECUTING A STANDALONE PROCEDURE

A standalone procedure can be called in two ways –

Using the **EXECUTE** keyword

Calling the name of the procedure from a PL/SQL block

The above procedure named '**greetings**' can be called with the EXECUTE keyword as –

EXECUTE greetings;

The above call will display –

Hello World

PL/SQL procedure successfully completed.

The procedure can also be called from another PL/SQL block –

BEGIN greetings; END;

/The above call will display –

Hello World PL/SQL procedure successfully completed.

DELETING A STANDALONE PROCEDURE

A standalone procedure is deleted with the **DROP PROCEDURE** statement. Syntax for deleting a procedure is –

DROP PROCEDURE procedure-name; You can drop the greetings procedure by using the following statement –

DROP PROCEDURE greetings;



EXAMPLE

```
SQL> create or replace procedure findMin(x IN number, y IN number, z OUT number) AS
```

```
2 BEGIN
3 IF x < y THEN
4 z:= x;
5 ELSE
6 z:= y;
7 END IF;
8 END;
9 /
```

Procedure created.

```
SQL> set serveroutput on
```

```
SQL> DECLARE
```

```
2 a number;
3 b number;
4 c number;
5 BEGIN
6 a:= 23;
7 b:= 45;
8 findMin(a, b, c);
9 dbms_output.put_line(' Minimum of (23, 45) : ' || c);
10 END;
11 /
```

Minimum of (23, 45) : 23

PL/SQL procedure successfully completed.

USING SAME PARAMETER FOR IN OUT

```
DECLARE
```

```
  a number;
```

```
PROCEDURE squareNum(x IN OUT number) IS
```

```
  BEGIN x := x * x;
```

```
END;
```

```
BEGIN
```

```
  a:= 23;
```

```
  squareNum(a);
```

```
  dbms_output.put_line(' Square of (23): ' || a);
```

```
END;
```

```
Square of (23): 529
```

```
PL/SQL procedure successfully completed.
```

CREATING A FUNCTION

A standalone function is created using the **CREATE FUNCTION** statement.

```
CREATE [OR REPLACE] FUNCTION function_name [(parameter_name [IN | OUT | IN OUT] type [, ...])] RETURN return_datatype {IS | AS}  
BEGIN < function_body >  
END [function_name];
```

The optional parameter list contains name, mode and types of the parameters. IN represents the value that will be passed from outside and OUT represents the parameter that will be used to return a value outside of the procedure.

The function must contain a **return** statement.

The *RETURN* clause specifies the data type you are going to return from the function.

function-body contains the executable part.

The AS keyword is used instead of the IS keyword for creating a standalone function.

Example: This function returns the total number of CUSTOMERS in the customers table.

We will use the CUSTOMERS table–

```
Select * from customers;
```

```
CREATE OR REPLACE FUNCTION totalCustomers RETURN number IS total number(2) := 0; BEGIN  
    SELECT count(*) into total FROM customers; RETURN total; END;
```

Function created.

CALLING FUNCTION

```
DECLARE c number(2);
```

```
BEGIN
```

```
c := totalCustomers();
```

```
dbms_output.put_line('Total no. of Customers: ' || c);
```

```
END;
```

/When the above code is executed at the SQL prompt, it produces the following result –

Total no. of Customers: 6

PL/SQL procedure successfully completed.



EXAMPLE

DECLARE

a number;

b number;

c number;

FUNCTION findMax(x IN number, y IN number) RETURN number IS z number; BEGIN

IF x > y THEN z:= x;

ELSE Z:= y;

END IF;

RETURN z;

END;

BEGIN

a:= 23; b:= 45;

c := findMax(a, b);

dbms_output.put_line(' Maximum of (23,45): ' || c);

END;Maximum of (23,45): 45

PL/SQL procedure successfully completed.

TRIGGERS

Triggers are stored programs, which are automatically executed or fired when some events occur. Triggers are, in fact, written to be executed in response to any of the following events –

A **database manipulation (DML)** statement (DELETE, INSERT, or UPDATE)

A **database definition (DDL)** statement (CREATE, ALTER, or DROP).

A **database operation** (SERVERERROR, LOGON, LOGOFF, STARTUP, or SHUTDOWN).

Triggers can be defined on the table, view, schema, or database with which the event is associated.

Benefits of Triggers

Triggers can be written for the following purposes –

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access
- Auditing
- Synchronous replication of tables
- Imposing security authorizations
- Preventing invalid transactions

TYPES OF TRIGGERS IN ORACLE

Triggers can be classified based on the following parameters.

Classification based on the **timing**

- BEFORE Trigger: It fires before the specified event has occurred.
- AFTER Trigger: It fires after the specified event has occurred.
- INSTEAD OF Trigger: A special type. You will learn more about the further topics. (only for DML)

Classification based on the **level**

- STATEMENT level Trigger: It fires one time for the specified event statement.
- ROW level Trigger: It fires for each record that got affected in the specified event. (only for DML)

Classification based on the **Event**

- DML Trigger: It fires when the DML event is specified (INSERT/UPDATE/DELETE)
- DDL Trigger: It fires when the DDL event is specified (CREATE/ALTER)
- DATABASE Trigger: It fires when the database event is specified (LOGON/LOGOFF/STARTUP/SHUTDOWN)

So each trigger is the combination of above parameters.

CREATING TRIGGERS

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF }
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW] WHEN (condition)
DECLARE
Declaration-statements
BEGIN
Executable-statements
EXCEPTION Exception-handling-statements
END;
```

This clause should be used based on the DML event. Below table will specify which clause is valid for which DML statement (INSERT/UPDATE/DELETE).

	INSERT	UPDATE	DELETE
:NEW	VALID	VALID	INVALID. There is no new value in delete case.
:OLD	INVALID. There is no old value in insert case	VALID	VALID

EXAMPLE

Customer(ID,Name,Age,Address,Salary)

CREATE OR REPLACE TRIGGER display_salary_changes

BEFORE DELETE OR INSERT OR UPDATE

ON customers

FOR EACH ROW WHEN (NEW.ID > 0)

DECLARE sal_diff number;

BEGIN sal_diff := :NEW.salary - :OLD.salary;

dbms_output.put_line('Old salary: ' || :OLD.salary);

dbms_output.put_line('New salary: ' || :NEW.salary);

dbms_output.put_line('Salary difference: ' || sal_diff);

END;

o/p:

Trigger created.

TRIGGERING A TRIGGER

Let us perform some DML operations on the CUSTOMERS table. Here is one INSERT statement, which will create a new record in the table –

```
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY) VALUES (7, 'Kriti', 22, 'HP', 7500.00);
```

When a record is created in the CUSTOMERS table, the above create trigger, **display_salary_changes** will be fired and it will display the following result –

Old salary: New salary: 7500 Salary difference:

Because this is a new record, old salary is not available and the above result comes as null.

Let us now perform one more DML operation on the CUSTOMERS table. The UPDATE statement will update an existing record in the table –

UPDATE customers SET salary = salary + 500 WHERE id = 2; When a record is updated in the CUSTOMERS table, the above create trigger, **display_salary_changes** will be fired and it will display the following result –

Old salary: 1500 New salary: 2000 Salary difference: 500

TRIGGERS:EXAMPLE

Let us again consider a table which is named STUDENT.

Let us now create a row-level trigger for the STUDENT table that would get executed by the DML statement like UPDATE, INSERT or DELETE on that table. The trigger will compute and show the age difference between current and previous values.

Code Implementation for trigger creation:

```
CREATE OR REPLACE TRIGGER age_changes
BEFORE DELETE OR INSERT OR UPDATE ON student
FOR EACH ROW
WHEN (NEW.CODE > 0)
DECLARE
    age_diff number;
BEGIN
    age_diff := :NEW.age - :OLD.age;
    dbms_output.put_line ('Prevoius age: ' || : OLD.age);
    dbms_output.put_line ('Current age: ' || : NEW.age);
    dbms_output.put_line ('Age difference: ' || age_diff);
END;
/
INSERT INTO STUDENT VALUES (4, 'MARY', 16, 97);
UPDATE student SET age = age + 1 WHERE code = 7;
```

TRIGGERS:EXAMPLE

For Example: The price of a product changes constantly. It is important to maintain the history of the prices of the products.

We can create a trigger to update the 'product_price_history' table when the price of the product is updated in the 'product' table.

1) Create the 'product' table and 'product_price_history' table

```
CREATE TABLE product_price_history (product_id number(5), product_name varchar2(32),  
supplier_name varchar2(32), unit_price number(7,2) );
```

```
CREATE TABLE product (product_id number(5), product_name varchar2(32),  
supplier_name varchar2(32), unit_price number(7,2) );
```

2) Create the price_history_trigger and execute it.

```
CREATE or REPLACE TRIGGER price_history_trigger BEFORE UPDATE OF unit_price ON  
product FOR EACH ROW BEGIN INSERT INTO product_price_history VALUES  
(:old.product_id, :old.product_name, :old.supplier_name, :old.unit_price); END; /
```

3) Lets update the price of a product.

```
UPDATE PRODUCT SET unit_price = 800 WHERE product_id = 100;
```

SEQUENCES

Sequence is a set of integers 1, 2, 3, ... that are generated and supported by some database systems to produce unique values on demand.

A sequence is a user defined schema bound object that generates a sequence of numeric values.

Sequences are frequently used in many databases because many applications require each row in a table to contain a unique value and sequences provides an easy way to generate them.

The sequence of numeric values is generated in an **ascending or descending order** at defined intervals and can be configured to restart when exceeds max_value.



Syntax:

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value

MINVALUE minimum value

MAXVALUE maximum value

CACHE| NOCACHE

CYCLE|NOCYCLE ;

sequence_name: Name of the sequence.

initial_value: starting value from where the sequence starts.

increment_value: Value by which sequence will increment itself. Increment_value can be positive or negative.

minimum_value: Minimum value of the sequence.

maximum_value: Maximum value of the sequence.

cycle: When sequence reaches its set_limit it starts from beginning.

nocycle: An exception will be thrown if sequence exceeds its max_value.

```
CREATE SEQUENCE sequence_1  
start with 1  
increment by 1  
minvalue 0  
maxvalue 100  
cycle;
```

```
CREATE SEQUENCE sequence_2  
start with 100  
increment by -1  
minvalue 1  
maxvalue 100  
cycle;
```

HOW TO USE

Sequence_name.CURRVAL: to access current value

Sequence_name.NEXTVAL: to access next value

```
CREATE TABLE students ( ID number(10), NAME char(20) );
```

```
INSERT into students VALUES(sequence_1.nextval,'Ramesh');
```

```
INSERT into students VALUES(sequence_1.nextval,'Suresh');
```

```
| ID | NAME |
```

```
| 1 | Ramesh |
```

```
| 2 | Suresh |
```

```
Select sequence_1.CURRVAL, sequence_1.NEXTVAL from dual;
```

```
DROP SEQUENCE sequence_name;
```

```
DROP SEQUENCE sequencesy;
```

```
ALTER SEQUENCE sequencesy
```

```
INCREMENT BY 2;
```



LINK FOR TEST

https://docs.google.com/forms/d/e/1FAIpQLSfiO5q2im7aG0MuPmJGvUU5TTvkeY0TxZqBIJoTBvHIAkT4aw/viewform?usp=sf_link

